lowThrust: how it works

A low-thrust (ion) vessel is flown from its current orbit to an orbit the user types in, `body: ap= pe= [i= lan= argpe=]`: about the same body, a moon, another planet or that planet's moon. The **solver** (`mission.py` with `guidance`, `transfer`, `orbit`, `bodies`) decides every tick from the current state; it runs in a patched-conic stand-in (`sim.py`) and, unchanged, in the game through the **pilot** (`pilot.py`), which flies it under rails warp with Persistent Thrust (PT) and this package's kRPC plugin **LowThrustWarp**.

1. The phase machine

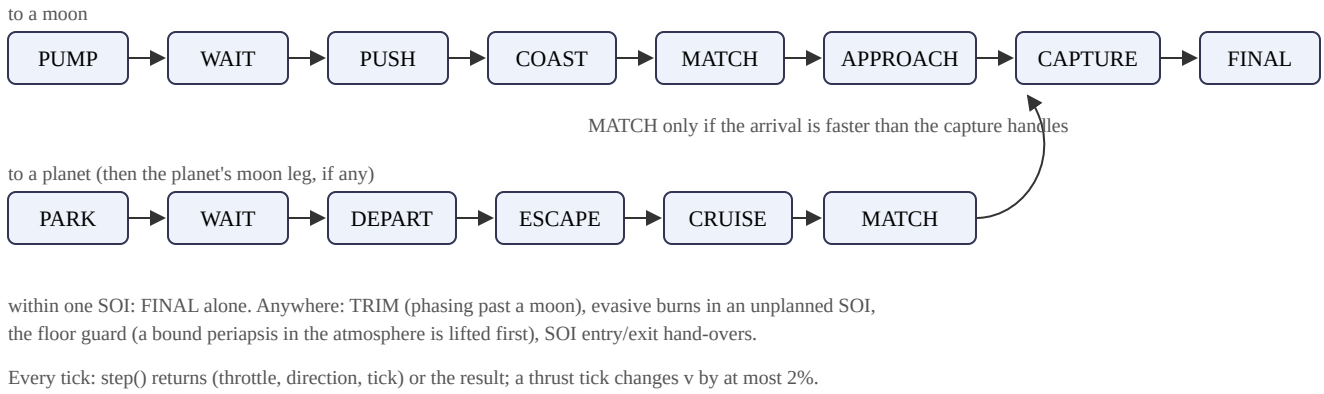


Figure 1. Phases and what triggers each transition.

Within one SOI (`guidance`, FINAL)

- Prograde/retrograde arcs centred on a fixed apse line (re-locked between burns), each arc fixing the opposite apsis; outward changes first; the plane change at the nodes on the biggest orbit; overlapping burns combined into one vector.
- Ticks are uniform in angle, not time. While the orbit crosses another moon's band the arcs are the widest (fewest revolutions there); each arc's first tick flies the arc in prediction by the law itself and caps it so the next revolution meets no moon; a phasing trim once a revolution slips a pass past a moon's SOI.

To a moon

- **PUMP** to a waiting orbit in the moon's plane: apoapsis 0.6 of the way, periapsis as low as lets the engine capture the arrival (one pass, or the long capture inside the SOI); placed in the gap between other moons' bands.
- **WAIT** for the push point (periapsis, or every 30 deg if it costs at most 1.1x the periapsis Hohmann) whose prograde push puts a direct encounter on the aim; decided far enough ahead to start half the burn early, then locked. **PUSH** flies it as the finite burn it is, re-solved.
- **COAST** with mid-course corrections on the aim error, measured at the SOI entry (patched conics: the entry state sets the periapsis). **MATCH**, when needed, sheds the excess speed about the planet by zero-effort-miss/velocity guidance onto a point just outside the SOI; ends 600 s before it or after twice the planned dv.
- **APPROACH** holds the periapsis target inside the SOI; **CAPTURE** (`transfer.capture_step`) only has to end bound: one pass centred on periapsis, or the long capture throttled at q = energy to shed / (a x path left), started at q = 0.8. **FINAL** shapes the orbit.

To a planet

- **PARK** in an ellipse whose periapsis push leaves on the porkchop window's asymptote; **DEPART** at the periapsis nearest the window (multi-pass if weak), each pass checked for moon hazards along the burn actually flown; **CRUISE** with B-plane corrections or a Lambert re-target; MATCH, APPROACH (dodges a harmful moon flyby), CAPTURE (polar aim at a planet with moons when the plane is free), FINAL.

2. The pilot in the game

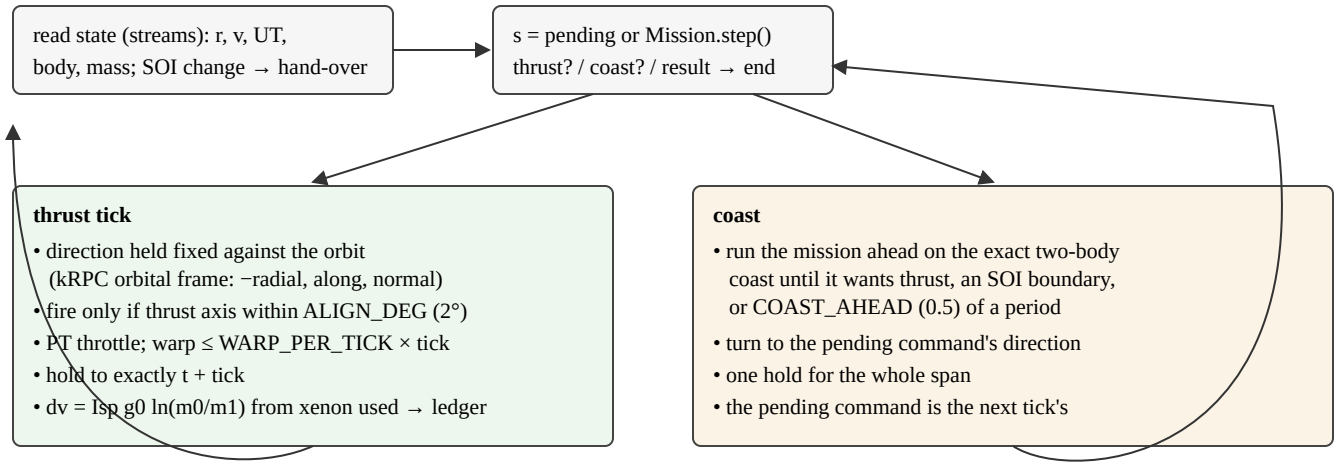


Figure 2. One pilot tick. Mass is the load mass less resources used (KSP freezes mass on rails).

- **Start**: the solar system is rebuilt from kRPC state vectors (`live.system`: elements per body, mean anomaly carried to UT 0; checked against `Orbit.position_at` 10^6 s ahead and logged). Frames: kRPC's non-rotating frame is left-handed with y north; the planner's swaps y and z. `prepare`: pace set, engines activated, SAS off, throttle 0, Isp read.
- **Panel**: target field, PLAN (the stand-in flies the mission from the live state as a preview), START, TERMINATE. Telemetry line per `LOG_INTERVAL_UT`: phase, body, ap, pe, i, true anomaly, angle from the held apse line, throttle fired/wanted, pointing error, tick, warp, mass, dv. The log ends with RPCs per tick and wall time per phase.
- **Shutdown** (arrival, failure, TERMINATE, crash): throttle 0, attitude cleared, hold released, pace 1, warp 0, SAS on.

3. The plugin (kRPC service `LowThrust`)



Figure 3. A hold: the warp rate and the pace through one tick, stopping within one physics step of its end.

procedure	what it does
SetThrottle, Throttle, Thrust, EngineCount	PT's throttle on every PersistentEngine (`SetThrottleAfterKey`) and ctrlState; kRPC's own throttle is zeroed by its warp. Set after going on rails.

RailsWarpRates, SetRailsWarp(i)	the game's rates (mods change them); the rate set at once, HIGH mode, capped by KSP's altitude limits. The index asked for stands: each frame the rate is the asked one as far as the altitude allows, so it climbs back after KSP lowers it. Cleared when the player warps or leaves rails.
SetPace, RunUntil(ut, floor), Held, ReleaseHold, AchievedRate	Unity timeScale = pace, limited so the next frame (the longest of the last 8 × 1.5, ≤ 0.04 s) runs at most half the steps left and under one step at the end; the rate steps down while 20 frames at it would pass the end, never below the floor index. At the end: timeScale 0 and kRPC served from the plugin's Update. Reading `Held` keeps a hold alive; 30 s unwatched and it is released (the pilot is gone).
SetAttitude(dir, frame), ClearAttitude, AttitudeError, AngularRate, WheelCharge, AngularAccelerationLimit	on rails the vessel is turned by its reaction wheels: torque (× authority) and MOI measured while unpacked; integrated in 0.02 s substeps; desired rate min(√(2·0.8·α·angle), angle/4 s), rate error closed over 0.5 s, inputs clamped ±1; EC drawn per |input| (a wheel short of EC loses torque); the thrust axis is the PT engine's up; the angular rate is handed to the rigidbodies on unpack. RCS and gimbal torque are not modelled.

Engine power is PT's: on rails it draws the engine's EC with its xenon (the Dawn: 8.74 EC/s at full thrust), and the vessel's generators must keep up.

4. Tunables (`config.py`, pilot)

field	default	meaning
ALIGN_DEG	2	a thrust tick fires only within this of the command
WARP_PER_TICK	5	warp rate ≤ this × tick length (s)
WARP_FLOOR_S	10	a hold steps down no lower than the rate for a tick this long
HOLD_LOST_S	2	wall s past a hold's time, not held, before it is asked again
RAILS_PACE	20	rails frames per real second × KSP's
COAST_AHEAD	0.5	longest coast run ahead in one hold, in periods
LOG_INTERVAL_UT	600	game s between telemetry lines